Project on Hamiltonian Simulation - Part II
Theory Exercises

Alessandro Cioffi (358488), Ameli Velasco Duran (414301)

Exercise 1: An Algorithm for Trotterised Hamiltonian Simulation

Exercise 1.1: Algorithm Sketch

Recall the Trotter formula:

$$e^{-iHt} \approx \left(\prod_P e^{-ia_P Pt/r} \right)^r \tag{1}$$

Given the helper function `qc.pauli(P, theta)` implementing $e^{iP\theta}$, consider the following pseudocode:

```
procedure trotter_approx(H, r, t):
    # H is a list of (coefficient, pauli_string) pairs;
    # initialise quantum circuit with n qubits,
    # with n = Pauli string length
    n = len(H[0][1])
    qc = quantum_circuit(n)

    for step in range(r):
        for (a_P, P) in H:
            theta = -a_P * t / r
            qc.pauli(P, theta)

    return qc
```

Explanations:

- The outer range(r) ensures the power of r is applied
- The product over P, or application of all gates, is done in the inner loop using the helper function
- For theta, we need $e^{-ia_P Pt/r}$, so we set $\theta = -a_P t/r$ so that $e^{iP\theta} = e^{iP(-a_P t/r)} = e^{-ia_P Pt/r}$

Exercise 1.2: Why Pauli Decomposition Format?

There is a dimensionality problem when providing the full n qubit Hamiltonian.
Since the Hamiltonian is a matrix of dimensions $2^n \times 2^n$, the number of entries to store is $O(4^n)$.
For large n (e.g. $n > 30$), this gets exponentially large and becomes impossible to store!

On the other hand, for a Pauli decomposition of length n, and a Hamiltonian H of poly(n) terms, we store $O(2 * poly(n) * n) = O(poly(n))$ terms. This is much more manageable, as it is polynomial in length (and not exponential)!

The circuit then has m $e^{-ia_P Pt/r}$ exponentials constructed with $O(n)$ gates, repeated r times, yielding complexity $O(mrn)$, polynomial in size. This is much better than the exponential complexity of the Hamiltonian derived earlier.

Exercise 2: The Transverse-Field Ising Hamiltonian

Recall the transverse-field Ising Hamiltonian formula:

$$H_{\text{Ising}} = -J \sum_{i=0}^{n-1} Z_i Z_{i+1 (\text{mod } n)} - h \sum_i X_i \tag{2}$$

2.1 Interaction Terms

Part (a): proof that $Z_i Z_j$ is diagonal

Lemma: The tensor product of diagonal matrices is diagonal.

Proof: Let $A = \text{diag}(a_1, \ldots, a_m)$ and $B = \text{diag}(b_1, \ldots, b_n)$ be diagonal matrices.
We write the Kronecker product $A \otimes B$ in block form:

$$A \otimes B = \begin{pmatrix} A_{11}B & A_{12}B & \cdots & A_{1m}B \\ A_{21}B & A_{22}B & \cdots & A_{2m}B \\ \vdots & \vdots & \ddots & \vdots \\ A_{m1}B & A_{m2}B & \cdots & A_{mm}B \end{pmatrix} \tag{3}$$

Since A is diagonal, $A_{ij} = 0$ for $i \neq j$, so all off-diagonal blocks vanish:

$$A \otimes B = \begin{pmatrix} a_1 B & 0 & \cdots & 0 \\ 0 & a_2 B & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & a_m B \end{pmatrix} \tag{4}$$

Since B is also diagonal, each block $a_i B$ is diagonal. Therefore, $A \otimes B$ is block-diagonal with diagonal blocks, making the entire matrix diagonal. □

Proof of part (a): By the lemma, since I and Z are diagonal, and Z_i, Z_j are tensor products of diagonal matrices, they are both diagonal in the computational basis.

Part (b): Action on computational basis states

We compute how $e^{i(Jt/r)Z_i Z_{i+1}}$ acts on basis states $|a\rangle_i |b\rangle_{i+1}$ for $a, b \in \{0, 1\}$.

First, recall that the Pauli Z matrix acts as:

$$Z|0\rangle = |0\rangle \tag{5}$$
$$Z|1\rangle = -|1\rangle \tag{6}$$

For the product $Z_i Z_{i+1}$:

$$(Z_i Z_{i+1}) |0\rangle_i |0\rangle_{i+1} = Z_i |0\rangle \otimes Z_{i+1} |0\rangle = |0\rangle \otimes |0\rangle = (+1)|00\rangle \tag{7}$$
$$(Z_i Z_{i+1}) |0\rangle_i |1\rangle_{i+1} = Z_i |0\rangle \otimes Z_{i+1} |1\rangle = |0\rangle \otimes (-|1\rangle) = (-1)|01\rangle \tag{8}$$
$$(Z_i Z_{i+1}) |1\rangle_i |0\rangle_{i+1} = Z_i |1\rangle \otimes Z_{i+1} |0\rangle = (-|1\rangle) \otimes |0\rangle = (-1)|10\rangle \tag{9}$$
$$(Z_i Z_{i+1}) |1\rangle_i |1\rangle_{i+1} = Z_i |1\rangle \otimes Z_{i+1} |1\rangle = (-|1\rangle) \otimes (-|1\rangle) = (+1)|11\rangle \tag{10}$$

The eigenvalues are $\lambda_{ab} = (-1)^{a+b}$: $+1$ when $a = b$, and -1 when $a \neq b$.

Applying the definition of the matrix exponential to $e^{i(Jt/r)Z_iZ_{i+1}}$ and using the eigenvalues computed above:

$$
\begin{aligned}
e^{i(Jt/r)Z_iZ_{i+1}} |a\rangle_i |b\rangle_{i+1} &= \sum_{k=0}^{\infty} \frac{(i(Jt/r))^k (Z_iZ_{i+1})^k}{k!} |a\rangle_i |b\rangle_{i+1} && (11)\\
&= \sum_{k=0}^{\infty} \frac{(i(Jt/r))^k (-1)^{k(a+b)}}{k!} |a\rangle_i |b\rangle_{i+1} && (12)\\
&= e^{i(Jt/r)(-1)^{a+b}} |a\rangle_i |b\rangle_{i+1} && (13)
\end{aligned}
$$

By substituting a and b, we get:

$$
\begin{aligned}
e^{i(Jt/r)Z_iZ_{i+1}} |0\rangle |0\rangle &= e^{i(Jt/r)} |0\rangle |0\rangle && (14)\\
e^{i(Jt/r)Z_iZ_{i+1}} |0\rangle |1\rangle &= e^{-i(Jt/r)} |0\rangle |1\rangle && (15)\\
e^{i(Jt/r)Z_iZ_{i+1}} |1\rangle |0\rangle &= e^{-i(Jt/r)} |1\rangle |0\rangle && (16)\\
e^{i(Jt/r)Z_iZ_{i+1}} |1\rangle |1\rangle &= e^{i(Jt/r)} |1\rangle |1\rangle && (17)
\end{aligned}
$$

2.2 Transverse Field Terms

Part (a): Diagonalizing basis

Recall the Pauli X matrix has eigenvectors:

$$
\begin{aligned}
|+\rangle &= \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle), \quad \text{eigenvalue } +1 && (18)\\
|-\rangle &= \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle), \quad \text{eigenvalue } -1 && (19)
\end{aligned}
$$

Applying the transverse field term $e^{i(ht/r)X_i}$ to each state:

$$
\begin{aligned}
e^{i(ht/r)X_i} |+\rangle &= \sum_{k=0}^{\infty} \frac{(i(ht/r)X_i)^k}{k!} |+\rangle && (20)\\
&= \sum_{k=0}^{\infty} \frac{(i(ht/r))^k X_i^k}{k!} |+\rangle && (21)\\
&= \sum_{k=0}^{\infty} \frac{(i(ht/r))^k \cdot 1^k}{k!} |+\rangle \quad (\text{since } X_i |+\rangle = |+\rangle\,) && (22)\\
&= \left(\sum_{k=0}^{\infty} \frac{(i(ht/r))^k}{k!} \right) |+\rangle && (23)\\
&= e^{i(ht/r)} |+\rangle && (24)
\end{aligned}
$$

Similarly:

$$e^{i(ht/r)X_i}\left|-\right\rangle = \sum_{k=0}^{\infty}\frac{(i(ht/r))^k X_i^k}{k!}\left|-\right\rangle \tag{25}$$

$$= \sum_{k=0}^{\infty}\frac{(i(ht/r))^k\cdot(-1)^k}{k!}\left|-\right\rangle \quad \text{(since } X_i\left|-\right\rangle = -\left|-\right\rangle\text{)} \tag{26}$$

$$= e^{-i(ht/r)}\left|-\right\rangle \tag{27}$$

Therefore $e^{i(ht/r)X_i}$ is diagonal in the X-basis $\{\left|+\right\rangle, \left|-\right\rangle\}$, with eigenvalues $e^{\pm i(ht/r)}$.

Part (b): Relation to rotation gate and bit flip

Recall the rotation gate's definition:

$$R_X(\theta) = e^{-i\theta X/2} \tag{28}$$

Compare with $e^{i(ht/r)X_i}$:

$$e^{i(ht/r)X_i} = e^{-i\theta X_i/2} \tag{29}$$

Matching exponents:

$$i(ht/r) = -i\theta/2 \quad \Rightarrow \quad \theta = -\frac{2ht}{r} \tag{30}$$

Therefore:

$$e^{i(ht/r)X_i} = R_{X_i}\left(-\frac{2ht}{r}\right) \tag{31}$$

When the rotation gate acts as a bit flip

For $e^{i(ht/r)X_i}$ to act proportionally to X_i, we need some scalar c such that:

$$e^{i(ht/r)X_i}\left|+\right\rangle = c\cdot X_i\left|+\right\rangle = c\left|+\right\rangle \tag{32}$$

$$e^{i(ht/r)X_i}\left|-\right\rangle = c\cdot X_i\left|-\right\rangle = -c\left|-\right\rangle \tag{33}$$

From part (a), this requires:

$$e^{i(ht/r)} = c \tag{34}$$

$$e^{-i(ht/r)} = -c \tag{35}$$

Substituting the first equation into the second:

$$e^{-i(ht/r)} = -e^{i(ht/r)} \iff 2\cos(ht/r) = 0 \iff ht/r = \frac{\pi}{2} + k\pi, \quad k\in\mathbb{Z} \tag{36}$$

Therefore:

$$t = \frac{\pi r(2k+1)}{2h}, \quad k\in\mathbb{Z}_{\geq 0} \tag{37}$$

Where we take $k\in\mathbb{Z}_{\geq 0}$ as $t\geq 0$ in the time evolution.

Exercise 3: Hamiltonian Simulation of the Transverse-Field Ising Hamiltonian in qBraid

Exercise 3.1: Circuit Implementation

See the jupyter notebook.

Exercise 3.2: Comparison Method

In order to compare two circuits for "closeness" (i.e. whether they are approximately the same), we can use distance defined through the operator norm:

$$D(U, V) = \|U - V\|_{\text{op}} \tag{38}$$

Where the operator norm $\|M\|_{\text{op}}$ is defined as $\max_{\||\psi\rangle\|=1} \|M|\psi\rangle\|_2$.

Indeed, if $U = V$, we have $D(U, V) = 0$ as required. Moreover, it is the same distance used in part 1. It is classical (i.e. extractable through `Operator(qc).data` in Qiskit), but sadly inefficient, as the full unitary scales exponentially with n.

Exercise 3.3: Comparison Results vs. Theoretical Bound

The theoretical bound for $n = 4$, $J = h = 1$, $t = 5$ and $m = 2n = 8$ is:

$$O\left(\frac{t^2}{r} \cdot m^2 \cdot \max_P |a_P|^2\right) = O\left(\frac{t^2}{r} \cdot 64 \cdot 1\right) = O\left(\frac{64t^2}{r}\right) \tag{39}$$

Expected error trend: since the theoretical bound scales with $1/r$, the error should decrease as $r \to \infty$.

Observed error trend: plotting the error defined in 3.2 reveals a decreasing trend, which aligns with the $1/r$ scaling.

Exercise 3.4(d): Interpretation of $p_0(t)$ Plots

The plots of $p_0(t)$ vary depending on h:

- $h = 0.2$: we have $h < J$, meaning the X_i terms don't affect the system as much; $p_0(t) \approx 1$ with a small Trotter error given that $\max_P |a_P|^2 = \max(J^2, h^2) = J^2 = 1$. Most r values align well with the exact solution, with only $r = 1, 2$ showing noticeable deviation.

- $h = 1$: we have $h = J$, which translates to considerable oscillations; there is strong divergence with small r (i.e. $r = 1, 2$), and overall alignment with the exact solution is not reached for most r values (except $r = 50$, which tracks the exact solution very well).

- $h = 2$: strong divergence at small r values, with significant and frequent oscillations. Likely caused by the term $\max(J^2, h^2) = h^2 = 4$ in the error bound. Similarly to $h = 1$, $r = 50$ tracks the exact solution closely.

Exercise 3.4(e): Is Fixed r Sensible?

No; as t increases with fixed r, the error bound $O(t^2/r \cdot m^2 \max_P |a_P|^2)$ grows boundlessly, scaling with t^2, resulting in a less accurate approximation. To counter this effect, we should choose $r \propto t^2$.

Exercise 3.4(f): Time-Dependent $r(t)$

Since we require $r(t) \propto t^2$ with $r(T) = r(5) = 30$ (as T = 5 according to the handout), we set:

$$r(t) = \left\lceil \frac{6}{5} t^2 \right\rceil \tag{40}$$

This way, the ceiling ensures getting an integer result, and $r(5) = \lceil \frac{6}{5} \cdot 25 \rceil = 30$. To avoid divisions by 0, we clip to $\max(1, \cdot)$ in the implementation.

Simulation results: plotting the simulation with time-dependent $r(t)$ shows more uniform alignment with the exact solution overall, with certain anomalies close to $r(t) = 1$. These results are noticeably better than those with fixed r, which shows using a time-dependent $r(t)$ is the better choice.

Exercise 3.5: Algorithm for approximating $\langle M \rangle_\psi$

To empirically approximate the magnetization

$$\langle M \rangle_\psi = \langle \psi | M | \psi \rangle,$$

we want to repeatedly measure the output state of the quantum circuit in the computational basis and estimate the expectation value from the measurement statistics.

Recall that the magnetization operator is

$$M = \sum_{i=0}^{n-1} Z_i.$$

Suppose the quantum circuit prepares the state

$$|\psi\rangle = \sum_{x \in \{0,1\}^n} \sqrt{p_x}\, |x\rangle,$$

where p_x is the probability of obtaining the bitstring x when measuring in the computational basis.

Since the computational basis states are eigenstates of each Z_i, the operator M is diagonal in the computational basis. Therefore, all cross terms vanish and we have

$$\langle M \rangle_\psi = \sum_{x \in \{0,1\}^n} p_x \langle M \rangle_x.$$

Therefore, the expectation value of the magnetization is simply the weighted average of the magnetizations of the measured bitstrings.

For a computational basis state x, the magnetization is computed by assigning:

$$0 \mapsto +1, \qquad 1 \mapsto -1,$$

for each qubit and summing over all qubits. For example,

$$\langle M \rangle_{|0000\rangle} = 4, \qquad \langle M \rangle_{|1111\rangle} = -4.$$

The empirical procedure is therefore:

1. Prepare the state $|\psi\rangle$ using the quantum circuit.

2. Measure all qubits in the computational basis many times.
3. Record the counts of each bitstring outcome.
4. Compute the magnetization associated to each measured bitstring.
5. Average these magnetizations weighted by their empirical probabilities.

If counts(x) denotes the number of times the outcome x was measured out of N total shots, then the empirical approximation is

$$\langle M \rangle_\psi \approx \sum_{x \in \{0,1\}^n} \frac{\text{counts}(x)}{N} \langle M \rangle_x.$$

As the number of shots increases, this empirical average converges to the true expectation value of the magnetization.

Exercise 3.6: Implementation of the magnetization estimator M_z

See the Jupyter notebook implementation.

Exercise 3.7: Time evolution of the total magnetization

See the Jupyter notebook implementation.

Exercise 4: Hamiltonian Simulation of the Transverse-Field Ising Hamiltonian in qBraid using real quantum device

Exercise 4.1: Trotterization of Time Evolution and Device Benchmarking

See the Jupyter notebook implementation.

For each value of $r \in \{1, 2, 3, 4, 5, 10\}$, we constructed the corresponding Trotterized quantum circuit approximating the evolution

$$|\psi_t\rangle = e^{iHt}|0^n\rangle,$$

with parameters $n = 4$, $J = 1$, $h = 0.2$, and $t = 2$.

Each circuit was executed on both an ideal simulator and the IQM Garnet quantum device accessed through qBraid, with 100 measurement shots for each execution. From the measurement counts, we estimated the probability p_0 of observing the all-zeros bitstring $|0000\rangle$ and plotted p_0 as a function of the number of Trotter steps r.

From the obtained graph, we first observe that the simulator and hardware results are relatively close for small values of r. In particular, between $r = 1$ and $r = 3$, both curves show an increase in p_0, indicating that the Trotterized approximation improves as the number of Trotter steps increases initially.

However, beyond approximately $r = 3$ or $r = 4$, the two curves begin to diverge more noticeably. On the ideal simulator, the probability p_0 remains very high, reaching values close to 1 around $r = 5$, and then decreases only slightly for larger values of r. This behavior suggests that the Trotter approximation is converging and that increasing r reduces the discretization error of the simulation.

In contrast, the results obtained on the IQM Garnet device behave differently after $r = 3$. Although the hardware curve initially follows the same trend as the simulator, it begins to decrease significantly for larger values of r. By $r = 10$, the measured probability has dropped to roughly 0.60, whereas the simulator still predicts a probability close to 0.96.

This discrepancy is likely caused by the accumulation of hardware noise as the circuits become deeper. Increasing the number of Trotter steps improves the theoretical accuracy of the approximation, but it also increases the number of quantum gates in the circuit. On a real quantum device, deeper circuits are more sensitive to decoherence, gate imperfections, and readout errors. As a result, the benefits of using larger r values are partially offset by the additional noise introduced during execution on the hardware.

Overall, these results illustrate the trade-off between Trotterization accuracy and hardware limitations in near-term quantum computing. While increasing the number of Trotter steps improves the approximation in the ideal simulation, the real quantum device eventually becomes dominated by noise effects, causing the experimental results to deviate from the ideal behavior.